

An Implementation of Ontology-Driven Reasoning for Conversational Question Answering over Knowledge Graphs

Kayra Özdemir and Hippolyte Le Carreres

Vrije Universiteit Amsterdam

Abstract. Large Language Models (LLMs) enable fluent conversational question answering, but often struggle to reliably enforce constraints and avoid hallucinations when operating without structured grounding. Knowledge graphs provide explicit domain structure and support logical reasoning, yet remain difficult to access through natural language interfaces. In this work, we present an implementation of an ontology-driven conversational question answering pipeline over a hotel-domain knowledge graph. Natural language user queries are translated into Manchester Syntax class expressions using an LLM, post-processed to correct semantic and syntactic inconsistencies, and executed over a reasoned knowledge graph enriched with a lightweight *TBox* ontology. The ontology encodes domain structure and property-chain axioms that enable inference over implicit relations such as geographic containment and proximity. Retrieved individuals are subsequently summarized into natural language responses using knowledge-graph-augmented generation. We evaluate the system on ten constructed dialogues (five simple, five complex) using error analysis and Intersection-over-Union (IoU) between ground-truth and generated query results, and we compare retrieved context to generated summaries. While post-processing generally yields well-formed queries, the summary stage still exhibits occasional omissions and hallucinated individuals, indicating that limitations in operator coverage, query generation and evaluation persist despite ontology-driven reasoning.

1 Introduction

LLMs support fluent conversational question answering but may omit constraints or hallucinate facts without grounding [10]. Knowledge graphs (KGs) offer structured domain knowledge, yet natural language access requires mapping user queries to representations and reasoning over implicit relations [7, 23].

We implement a logic-enhanced conversational QA pipeline over a hotel-domain KG by augmenting the *ABox* with a *TBox* ontology encoding domain structure and property-chain axioms (e.g., inferring `inCity`, `inCountry`, etc.) from `location`). Natural language questions are translated into Manchester Syntax using *oLlama*, post-processed, executed via an OWL reasoner, and summarized into responses through KG-augmented generation [8, 9, 14, 15, 18, 23].

2 Data

To construct our ontology, essentially the *TBox* to our *ABox*, we mostly utilized **property chains** [9]. In the classes we added three super classes: **Accommodation**, **BodyOfWater**, and **PublicTransport**. These were introduced to accommodate for more general constructs in user dialogues. The subclasses of these superclasses follow naturally. **BodyOfWater** serves as a superclass for all the *WikiData* classes (all the *Q*-numbers) [26]. The **Accommodation** serves as super class for **Hostel**, **Hotel**, and **Camping_Site**. **PublicTransport** is the super class of **Trainstation** [19, 23]. The ontology was designed in Protégé [22].

Besides these new classes, we added some additional subclass hierarchies. For all the **BodyOfWater** subclasses: e.g. `Gulf owl:subClassOf Bay`, `Gold River owl:subClassOf River`, etc., subclass hierarchies were added where semantically relevant. These hierarchies can be seen in the appendix, in Figure 1 [19, 23].

Most of our ontology modeling takes place in the object properties. We added domains and ranges to properties where their usage in the *ABox* was explicit as such [19]. For instance, the property `diet` is only used for **Restaurant** objects and **Diet** subjects. The most interesting ontology modeling, and most relevant to our project, lies in the property chains modeled for the properties: `nextTo`, `inCity`, `inCountry`, `location`, and `walkingDistance` (this last property was our addition). The data in our *ABox* is comprised of a lot of (mainly) accommodations that have a `location` triple, meaning that their location might be a specific **neighbourhood** in *Marseille* (e.g. `accommodation445`). However, these individuals do not have an explicit `inCity`, `inCountry`, or `nextTo` triple. The **neighbourhood** individuals have sufficient information that we might be able to say for `accommodation445`, that it is in the city *Marseille*, it is in the country of *France*, and it is located next to the *Gulf of Lion*. These links are not there explicitly, and require some ontology modeling to make [19].

The only information we have on a case like `accommodation445`, is that it has the location of this specific neighbourhood. Therefore, we utilize property chains to link all this information to our `accommodation445`. The property chains we implemented, and which allow us to infer much information about an individual

just via its location triple, are the following:

$$\begin{aligned} \text{location} \circ \text{nextTo} &\sqsubseteq \text{nextTo}, & \text{inCountry} \circ \text{nextTo} &\sqsubseteq \text{nextTo}, \\ & & \text{inCity} \circ \text{nextTo} &\sqsubseteq \text{nextTo} \end{aligned}$$

This allows us to entail that if `accommodation445` is in a neighbourhood of *Marseille*, it is in next to whatever is in near this neighbourhood of *Marseille* [9]. Next, we make similar chains for `inCity`, and `inCountry` [9]:

$$\begin{aligned} \text{location} \circ \text{inCity} &\sqsubseteq \text{inCity}, & \text{location} \circ \text{inCountry} &\sqsubseteq \text{inCountry}, \\ & & \text{inCity} \circ \text{inCountry} &\sqsubseteq \text{inCountry} \end{aligned}$$

Our final, property modeling takes place in the `walkingDistance` property. Here we make the assumption that if two individuals have the same location, i.e. if `accommodation445` is in the "*12th Arrondissement Of Marseille*" via a location triple, and some other entity would be in this neighbourhood, that these two individuals should be within walking distance of each other. This avoids having to set some arbitrary number (say, a km distance) that would conclude walkability, and require us to fetch geo-location of these two locations. Instead, we make a simple assumption. However, to say that these two entities are in walking distance of each other, we need more. To this end, we created a inverse property, `locationOf`, which has the `owl:inverseOf` property to `location`. Then we make a recognizable property chain axiom:

$$\text{location} \circ \text{locationOf} \sqsubseteq \text{walkingDistance}$$

This axiom says that if `accommodation445` is in the "*12th Arrondissement Of Marseille*", and this neighbourhood is the location of `accommodation476`, then `accommodation445` is in walking distance of `accommodation476` (and vice versa). It is not necessary to make `walkingDistance` symmetric, as this behaviour arises naturally from the the *ABox* and the ontology modeling [19].

One specific challenge that required a patchy fix, was the way in which `userRating` was modeled initially. The `userRating` property has as range, instances. It is an object property. For querying one might think that being able to work with literals, such as integers, would facilitate queries such as, hotel rated more than 3 stars. However, we kept this as is and instead added another super class: `HighRating`. This class is a subclass of `UserRating` and provides a way to query for highly rated accommodations. Here we used an equivalence axiom [23]:

$$\text{HighRating} \equiv \{4_stars, 5_stars\}$$

The dataset was further analyzed to obtain an overview of its size, structure, and modeling characteristics [1, 5, 7, 19]. Table 2 summarizes the global statistics of the constructed knowledge graph: the graph consists of 9,237 triples, connecting 1,789 unique subjects and 870 unique objects; resulting in 1,792 distinct URI nodes. Table 3 reports the frequency of predicate usage. The graph is strongly structured around `rdf:type`, which accounts for the largest share of

triples, reflecting extensive explicit typing of individuals. Beyond type assertions, a small set of domain-specific predicates (e.g. `ns1:hasFacility`, `ns1:location`, `ns1:userRating`) dominates the relational structure. This indicates that the dataset emphasizes descriptive and contextual relationships over complex schema-level axioms, shown by the low amount of predicates such as `rdfs:subClassOf`, `rdfs:domain`, and `rdfs:range`. Literal usage is analyzed in Table 4. All literal objects occur exclusively under the predicate `rdfs:label`, implying that literals are used solely for human-readable naming rather than for numerical values or typed attributes. Finally, Table 5 provides a detailed view of how instances are distributed across classes. These tables are presented in the appendix.

3 Methodology

3.1 Reasoner

We use the **SyncReasoner** from the *OWLAPY Python* library [2, 6, 12]. It allows us to use *Pellet's* [21] reasoning capabilities and is quite easy to deploy. We can then query against our reasoned knowledge graph using complex OWL class expressions [23].

3.2 Question to Query Conversion

To ensure the quality of LLM outputs is consistent, we assign a role to the LLM, in this case a specialized NLP engine that converts natural language questions to Manchester Syntax queries. We set some styling and syntax rules. Importantly, we add references of the classes, properties, and instances-only the frequently occurring instances. This improves the consistency of outputted queries [8, 14, 20, 25]. *oLlama* is also provided with 7 examples of dialogue-query pairs which include a broad range of reference constructs [18].

3.3 Query Post-Processing

The initial generated query goes through a few post-processing steps to ensure it is correctly formatted before querying the reasoned knowledge graph. We have a few helper functions that are necessary to ensure we can write a satisfiable OWL class expression. We utilize a second LLM generator in case our initial resolved and fixed query yields no instances. As it does not perform a novel role in the pipeline, further details are omitted (and can be consulted in the Github repository) [2, 13, 20, 23]. The helper functions will be explained along with the example below. The example dialogue considered here is dialogue 6 (see Table 6 in the appendix).

1. **Converted question to query:** Hotel and (hasFacility some Sauna) and (hasFacility some Swimming_Pool) and (inCountry value Portugal) and (nextTo some TrainStation)

Our knowledge graph consists of two namespaces, the first is our local ontology and the second is the wikidata entity namespace. The *Wikidata* entities, both classes and instances, are *Q*-numbers. To map each *Wikidata*

URI to some name would not be feasible. Each *Wikidata* instance has an `rdfs:label`. We convert entities to their *Wikidata* URI after the initial query is generated. The two helper functions in our code that handle this are the `GET_LOCAL_URI` and `RESOLVE_MANCHESTER_FILLERS` functions (see section 2) [8, 13, 26]. Entities from our local namespace do not need to be mapped to their full URIs, this is handled in the Manchester Syntax parser. Below is the resulting query.

2. **Resolved Wikidata URIs:** `Hotel` and `(hasFacility some Sauna)` and `(hasFacility some Swimming_Pool)` and `(inCountry http://www.wikidata.org/entity/Q45)` and `(nextTo some TrainStation)`
 We check for incorrect cases of *some* and *value* usage in the query. We want to avoid looking for an `OWLHasObjectValue` construct that has a class filler. Below the *some* and *value* are corrected [1, 2, 23, 26].
3. **Fixed *some*/*value* construct errors:** `Hotel` and `(hasFacility value Sauna)` and `(hasFacility value Swimming_Pool)` and `(inCountry value http://www.wikidata.org/entity/Q45)` and `(nextTo some TrainStation)`
 Lastly, we pass our resolved and fixed query into our Manchester query parser. We chose not to use the `MANCHESTER_TO_OWL` method from *OWLAPY* as it was limited in our use case [2, 8, 12]. It takes a single name space as input. As we deal with two namespaces in nearly all of our queries, we need to be able to convert some query with mixed namespaces into a single, correct OWL expression [23]. Some operators like `NOT` and `OR` were left out. Furthermore, we avoid nested queries to not break the parser. Consult our *Jupyter notebook GitHub repository* [1, 2, 12, 13] for an example test case of the reasoner which compiles all the above highlighted functionality.
4. **Parsed and converted to OWL expression:**
`OWLObjectIntersectionOf(...), OWLHasObjectValue(...), or`
`OWLSomeValuesFrom(...)`

3.4 Knowledge to Summary/Output Conversion

For the final summary response, the choices made are: specify the role, avoid a fixed output format, and emphasis on avoiding hallucinations. An initial implementation of the instruction set for this part of the pipeline enforced a stricter output format, and had a different role assignment for the LLM. This resulted in outputs less conform with a natural language output. To consult these previous outputs, look through our *Jupyter local or Colab notebook* version **one**. The LLM is also provided one example case [10, 12–14, 20, 25].

4 Experimental Setup

4.1 Example Dialogues

For the first 5 dialogues we explore questions with the upper limit of constructs (i.e. 2 or 3). In the 5 complex dialogues we test dialogues of four, five, and six constructs. This allows us to test the limits of the question to query conversion and our own query parsing pipeline by having large conjunctions of constructs.

We also introduce lexical variation in the dialogues by using different natural language expressions (e.g., “near” or “right by”) that all map to the same ontology property (**nextTo**). We want the model to be able to fit natural language to one of our existing properties and not hallucinate properties that do not exist; questions that ask for specific countries and **nextTo** constructs. These allow us to test our property chains modeled in the ontology [9, 15, 16, 24]. We also have a dialogue with walking distance as a construct. Dialogues are also constructed with testing class or individual filler performance. That is, to see to what extent LLMs can properly map **OWLHasValueFrom** and **OWLSomeIndividualsFrom** constructs. These two objects are modeled in the *OWLAPY* library. Generally, the dialogues are constructed to test for large conjunctions [4, 24]. These dialogues are displayed in the appendix, in Table 6.

4.2 Evaluation Metrics

Three main evaluation strategies were used: 1) comparison of the ground truth queries and the initial LLM-generated queries. We evaluate the quality of the initial query generation without post-processing by measuring the number of syntactic, semantic, or mapping errors. 2) Intersection-over-Union (IoU) [11] between the instances retrieved by the ground truth queries and the LLM-generated queries, to evaluate the performance of the LLM query generation. 3) Human review of the final summarized responses supported by IoU scores between the retrieved context and the GraphRAG output, to evaluate the general quality and truth of the final summary output [5, 10, 15–17].

5 Results

Table 1A presents the error analysis of initial LLM-generated queries prior to post-processing. Most dialogues contain no errors, indicating generally well-formed Manchester Syntax output. Observed errors mainly involve incorrect *some/value* usage, missing constructs, or occasional property mismappings (e.g., **walkingDistance** vs. **nextTo**). These errors motivate the lightweight post-processing and correction steps in our pipeline [8, 11, 23].

For the IoU scores between ground-truth and generated queries, performance was strong overall: only one dialogue retrieved no instances ($IoU = 0$), while the remaining nine achieved perfect overlap ($IoU = 1$; Table 1B) [11].

In contrast, IoU scores between the retrieved context and the final GraphRAG-generated summaries exhibit greater variation (Table 1C). Three of the ten dialogues show discrepancies between retrieved individuals and those referenced in the final responses. Notably, there is no clear correlation between the total number of retrieved instances and the number of omitted instances: dialogue 4 retrieved the largest number of instances yet achieved an IoU score of 1, indicating no omissions. Two cases of hallucination were observed. In dialogue 1, no individuals matched the query, yet the GraphRAG response mentioned several hotels that were not present in the retrieved context. Similarly, in dialogue 6, the LLM hallucinated alternative hotels despite no retrieved instances. Apart from

these cases, no additional hallucinated individuals were observed; all omissions are reported in Table 1C [10, 11, 15–17].

Table 1. Evaluation results for query accuracy and retrieval performance.

A. Error Analysis of LLM-Generated Queries				
Dialogue	Errors	Notes		
1, 3, 5, 6, 10	0	No errors detected		
2	1	Incorrect <i>some/value</i> mapping		
4	1	Incorrect <i>some/value</i> mapping		
7	2	Incorrect <i>some/value</i> mapping; missing construct		
8	1	Incorrect property (nextTo vs. walkingDistance)		
9	1	Missing construct (nationality value French)		

B. Query IoU and Differences			C. GraphRAG IoU and Omissions			
Dialogue	IoU	Difference	Diag.	IoU Retr.	Omitted	Instances
1–7, 9, 10	1.0	$\emptyset$	1, 6–9	1.0	0–3	$\emptyset$
8	0.0	{acc. 524}	2	1.0	16	$\emptyset$
			4	1.0	196	$\emptyset$
			3	0.67	9	{rest. 279, acc. 542, 437}
			5	0.70	23	{7 accommodations}
			10	0.27	11	{8 accommodations}

6 Discussion

Our pipeline robustly handles large conjunctions, for instance in dialogue 7 with six constructs. We extend *OWLAPY*’s capabilities with custom helper functions to achieve Manchester-to-OWL structures, enabling reliable parsing of complex queries [2, 8]. A third *oLLama* agent was introduced to repair invalid or empty initial queries, successfully correcting certain mapping errors (e.g., **walkingDistance** misinterpreted as **nextTo**); however, due to LLM stochasticity, additional agents did not consistently improve overall query quality [3, 18].

Given time constraints and limitations of *OWLAPY*, we did not support operators such as **NOT** and **OR**, and avoided nested expressions by modeling them as additional conjuncts [1, 2, 23]. The implications of this decision for query and OWL expression complexity require further study. Prioritizing natural language relaxes constraints, increasing the risk that LLMs will hallucinate elements or mismap constructs when encountering unseen information.

Finally, the current IoU evaluation captures omissions but not hallucinated individuals, suggesting the need for a more comprehensive evaluation metric [5, 10, 11, 15, 16].

References

1. Baader, F., Calvanese, D., McGuinness, D.L., Nardi, D., Patel-Schneider, P.F. (eds.): The Description Logic Handbook: Theory, Implementation, and Applications. Cambridge University Press (2003), https://assets.cambridge.org/052178/1760/frontmatter/0521781760_frontmatter.pdf, front matter available at https://assets.cambridge.org/052178/1760/frontmatter/0521781760_frontmatter.pdf
2. Baci, A., Friedrichs, L., Demir, C., Ngomo, A.C.N.: Owlapy: A pythonic framework for owl ontology engineering. arXiv preprint arXiv:2511.08232 (2025)
3. Bender, E.M., Gebru, T., McMillan-Major, A., Shmitchell, S.: On the dangers of stochastic parrots: Can language models be too big? In: Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency. pp. 610–623. ACM (2021). <https://doi.org/10.1145/3442188.3445922>, <https://doi.org/10.1145/3442188.3445922>
4. Budzianowski, P., et al.: Multiwoz – a large-scale multi-domain wizard-of-oz dataset for task-oriented dialogue modelling. arXiv preprint arXiv:1810.00278 (2018)
5. Gangemi, A., Catenacci, C., Ciaramita, M., Lehmann, J.: Ontology evaluation and validation. In: Handbook on Ontologies, pp. 297–318. Springer (2009), ontology quality assessment and validation methods
6. Glimm, B., Horrocks, I., Motik, B., Stoilos, G., Wang, Z.: Hermit: An owl 2 reasoner. Journal of Automated Reasoning **53**(3), 245–269 (2014), system description of the HermiT OWL 2 reasoner
7. Höffner, K., Walter, S., Marx, E., Usbeck, R., Lehmann, J., Ngonga Ngomo, A.C.: Survey on challenges of question answering in the semantic web. In: Semantic Web Evaluation Challenges, pp. 1–20. Springer (2016), includes knowledge graph statistics and profiling practices
8. Horridge, M., Patel-Schneider, P.F.: Owl 2 web ontology language: Manchester syntax. <https://www.w3.org/TR/owl2-manchester-syntax/> (2012), w3C Recommendation
9. Horridge, M., Patel-Schneider, P.F.: Owl 2 web ontology language primer. https://www.w3.org/TR/owl2-primer/#Property_Chains (2012), w3C Recommendation
10. Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., Liu, T.: A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. arXiv preprint arXiv:05232 (2024), accepted by ACM Transactions on Information Systems (TOIS). Last revised: 19 Nov 2024. <https://doi.org/10.48550/arXiv.05232>
11. Jaccard, P.: The distribution of the flora in the alpine zone. New Phytologist **11**(2), 37–50 (1912), origin of the Jaccard Index / Intersection over Union
12. Lamy, J.B.: Owlready2: Ontology-oriented programming in Python. <https://pypi.org/project/Owlready2/> (2017), accessed: 2024-05-20
13. Le Carreres, : Source code for PSAI2026 Project. <https://github.com/HippolyteLC/Project-Situated-AI-2026.git> (2026), accessed: 15-01-2026
14. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive nlp tasks. arXiv preprint arXiv:2005.11401 (2020), <https://doi.org/10.48550/arXiv.2005.11401>
15. Li, Z., Chen, Z., Zhao, I., Durme, B.V., Gardent, C., Van Durme, B.: Faithfulqa: A benchmark for faithfulness in question answering. arXiv preprint arXiv:2202.03629 (2022), accessed: 2026-01-xx

16. Maynez, J., Narayan, S., Bohnet, B., McDonald, R.: On faithfulness and factuality in abstractive summarization. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. pp. 1906–1919. Association for Computational Linguistics, Online (2020). <https://doi.org/10.18653/v1/2020.acl-main.173>, <https://aclanthology.org/2020.acl-main.173/>
17. Min, S., et al.: Factscore: Fine-grained atomic evaluation of factual precision in generative models. arXiv preprint arXiv:2305.14251 (2023)
18. Ollama: Ollama: Run large language models locally. <https://ollama.com/> (2024), accessed: 2026-01-15
19. Ontology Design Patterns Initiative: Ontology design patterns. <http://ontologydesignpatterns.org/> (2024), accessed: 2026-01-15
20. Sahoo, P., Singh, A.K., Saha, S., Jain, V., Mondal, S., Chadha, A.: A systematic survey of prompt engineering in large language models: Techniques and applications. arXiv preprint arXiv:2402.07927 (2024), <https://doi.org/10.48550/arXiv.2402.07927>
21. Sirin, E., Parsia, B., Grau, B.C., Kalyanpur, A., Katz, Y.: Pellet: A practical owl-dl reasoner. *Journal of Web Semantics* **5**(2), 51–53 (2007)
22. Stanford Center for Biomedical Informatics Research: Protégé: An open-source ontology editor and framework. <https://protege.stanford.edu/> (2024), accessed: 2026-01-15
23. W3C OWL Working Group: Owl 2 web ontology language overview. <https://www.w3.org/TR/owl2-overview/> (2012), w3C Recommendation
24. Wang, Y., Berant, J., Liang, P.: Building a semantic parser overnight. In: Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). Association for Computational Linguistics (2015), <https://aclanthology.org/P15-1129/>
25. Wei, J., et al.: Finetuned language models are zero-shot learners. arXiv preprint arXiv:2109.01652 (2022)
26. Wikidata Contributors: Wikidata: A free and open knowledge base. <https://www.wikidata.org/> (2024), accessed: 2026-01-15

A Appendix

Table 2. Basic statistics.

Statistic	Count
Triples	9,237
Unique subjects	1,789
Unique predicates	14
Unique objects	870
Distinct classes*	32

Table 4. Literal statistics.

Statistic	Value
Literals	485
Avg. length	13.39

Table 3. Predicate frequency.

Predicate	Count
<code>rdf:type</code>	3,751
<code>ns1:hasFacility</code>	1,640
<code>ns1:location</code>	1,085
<code>ns1:userRating</code>	1,000
<code>rdfs:label</code>	485
<code>ns1:nationality</code>	472
<code>ns1:restaurantType</code>	305
<code>ns1:inCity</code>	267
<code>ns1:diet</code>	137
<code>rdfs:subClassOf</code>	18

*Only classes appearing in at least one `rdf:type` assertion are counted.

Table 5. Distribution of instances across classes in the knowledge graph.

Rank	Class	Instances	Rank	Class	Instances
1	<code>owl:NamedIndividual</code>	1,746	17	<code>ns1:UserRating</code>	5
2	<code>ns1:Restaurant</code>	506	18	<code>wd:Q573344 (label: Main Stream)</code>	5
3	<code>ns1:Hotel</code>	344	19	<code>ns1:Country</code>	5
4	<code>ns1:Camping_Site</code>	342	20	<code>ns1:Diet</code>	3
5	<code>ns1:Hostel</code>	314	21	<code>wd:Q165 (label: Sea)</code>	2
6	<code>ns1:Neighbourhood</code>	267	22	<code>wd:Q166620 (label: Drainage Basin)</code>	2
7	<code>ns1:Museum</code>	55	23	<code>wd:Q12284 (label: Canal)</code>	2
8	<code>ns1:City</code>	34	24	<code>wd:Q125941113 (label: Gold River)</code>	2
9	<code>owl:Class</code>	33	25	<code>wd:Q1973404 (label: Adjacent Sea)</code>	2
10	<code>ns1:Trainstation</code>	22	26	<code>wd:Q39594 (label: Bay)</code>	2
11	<code>wd:Q4022 (label: River)</code>	12	27	<code>owl:Ontology</code>	1
12	<code>ns1:Nationality</code>	10	28	<code>wd:Q1322134 (label: Gulf)</code>	1
13	<code>owl:ObjectProperty</code>	9	29	<code>wd:Q355304 (label: Watercourse)</code>	1
14	<code>ns1:Facility</code>	8	30	<code>wd:Q986177 (label: Mediterranean Sea)</code>	1
15	<code>ns1:TouristAttraction</code>	8	31	<code>wd:Q9430 (label: Ocean)</code>	1
16	<code>ns1:RestaurantType</code>	5	32	<code>wd:Q187223 (label: Lagoon)</code>	1
—	<code>ns1:Accommodation</code>	0	—	<code>ns1:Airport</code>	0
—	<code>ns1:BodyOfWater</code>	0	—	<code>ns1:PublicTransport</code>	0

Table 6. Dialogues and corresponding queries (#1-5 simple, #6-10 complex).

ID	Dialogue (String)	Formal Query (Manchester Syntax)
1	"Find me an accommodation that is in Portugal and near a landmark."	Accommodation and (inCountry value Portugal) and (nextTo some Landmark)
2	"I want to go to a camping by a river which is rated 5 stars."	Camping_Site and (nextTo some River) and (userRating value 5_stars)
3	"What are Arabic restaurants that serve fast food?"	(nationality value Arabic) and Restaurant and (diet value FastFood)
4	"Find me all accommodations rated 5 stars."	Accommodation and (userRating value 5_stars)
5	"Hotels near the sea."	Hotel and (nextTo some Sea)
6	"I want a hotel with a sauna and a swimming pool, preferably in Portugal near a train station."	Hotel and (hasFacility value Sauna) and (hasFacility value Swimming_Pool) and (inCountry value Portugal) and (nextTo some TrainStation)
7	"Find me a hotel that is also a Michelin restaurant in France. They should have free parking and a 24 hour front desk too."	Hotel and (restaurantType value Michelin) and Restaurant and (inCountry value France) and (hasFacility value Free_Parking) and (hasFacility value 24h_front_desk)
8	"Can you find me a hostel in Porto with an airport shuttle. It would be nice if it is within walking distance from a museum."	Hostel and (inCity value Porto) and (hasFacility value Airport_Shuttle) and (walkingDistance some Museum)
9	"I want a French Restaurant in Turkey right by the sea with free Wi-Fi."	(nationality value French) and Restaurant and (inCountry value Turkey) and (nextTo some Sea) and (hasFacility value Free_Wifi)
10	"I want a hotel that has a restaurant with private bathrooms. Preferably rated highly."	Hotel and (hasFacility value RestaurantInHotel) and (hasFacility value Private_Bathroom) and (userRating some HighRating)

Fig. 1. Class hierarchy.